A Ablation Study

A.1 Edge Metric Comparison

We evaluated SSONN's targeted expansion against two classes of baselines: (i) reversed-gradient criteria that expand at low-sensitivity edges, and (ii) magnitude-based criteria that select edges by ℓ_1 or ℓ_2 weight values. All models are trained on MNIST for 64 steps, with a single expansion triggered at step 10. Figure 2 presents the validation accuracy trajectories.

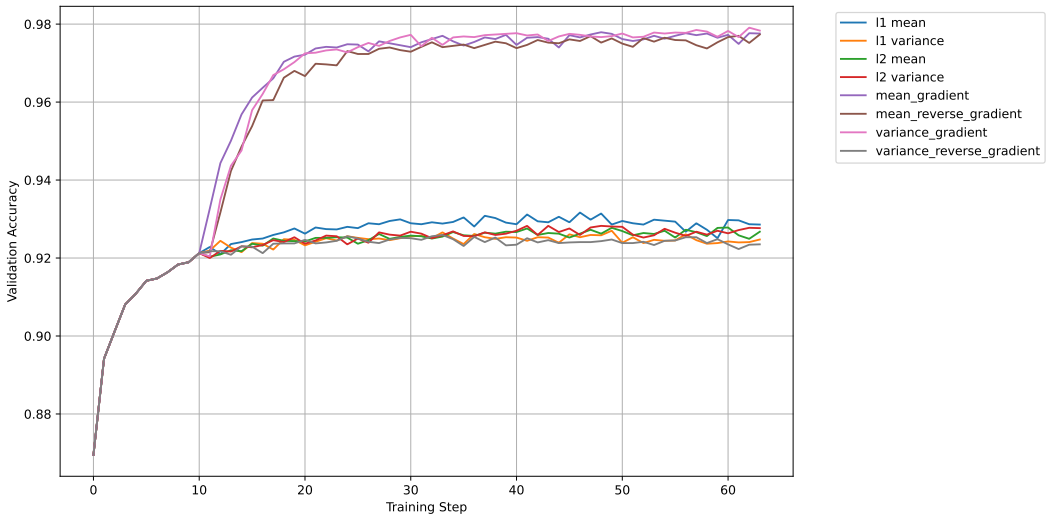


Figure 2: Validation accuracy for different metrics on MNIST

Immediately after the expansion event, both of the high-sensitivity criteria g_{abs} and g_{var} show a dramatic increase in precision from about 0.92 to 0.98. That indicates that the new capacity injected at the edges where the loss is most responsive produces substantial representational gains. In contrast, the reversed gradient baselines incur negligible improvement. Expanding at connections to which the loss is insensitive does not produce meaningful new learning directions. This failure is not just empirical but rooted in a theoretical inconsistency. Low-gradient edges correspond to near-flat regions of the loss surface, so appending sublayers there cannot break existing linearity; we formalize this argument in Subsection C.

Magnitude-based expansions (ℓ_1 and ℓ_2 weight norms) similarly underperform, delivering at best a 0.93 plateau. Furthermore, gradient-targeted models converge more rapidly: by step 20, the mean gradient variant surpasses 0.97, while all other methods lag by at least 3%. These results confirm that SSONN's principle of injecting capacity at high-sensitivity edges is both theoretically justified and practically effective.

A.2 Comparison with GradMax

Figure A.1 shows the training loss trajectories for SSONN and several competing methods, including GradMax [8]. Unfortunately, we were unable to exactly reproduce their results from the available code. Instead, we ran an experiment in the same training regime for our model, using an MLP of a smaller size to demonstrate that any implementation advantage lies in the selection strategy, rather than the raw capacity.

GradMax [8] developing ideas from FireFly [32] starts with a 100:25:10 student architecture, aiming to achieve the results of a bigger pre-known 100:50:10 teacher architecture. Despite starting from a smaller network (100:10 MLP), SSONN achieves sub-unit loss in under 500 steps, more than seven times faster than both GradMax and all other baselines, which only approach a loss of 1.0 after $3{,}500$ steps. Thus, SSONN's targeted sublayer insertion at high-sensitivity edges unlocks representational power far more efficiently than existing criteria, even when those criteria are applied to larger models.

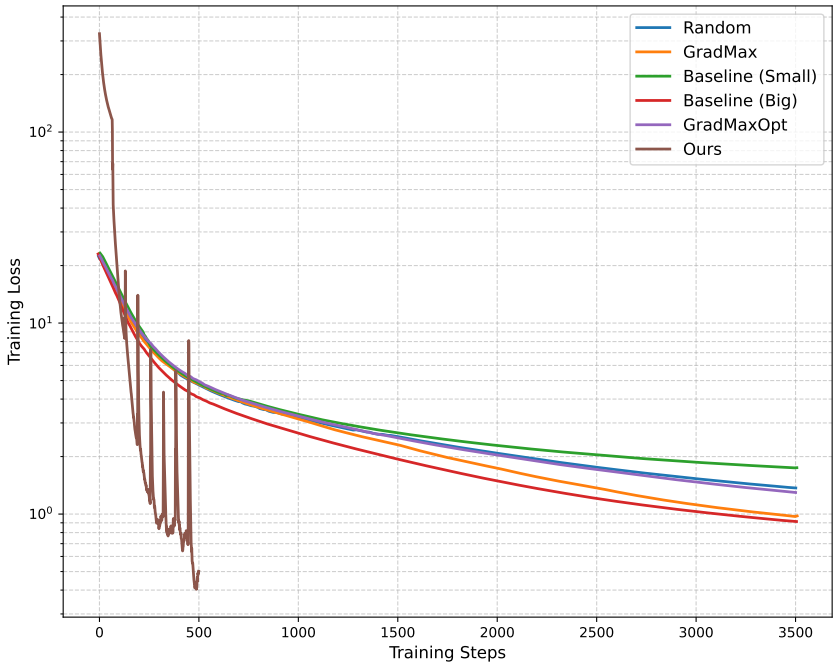


Figure 3: Comparison of SSONN and other expansion methods

B Hyperparameters

The dynamic expansion in SSONN relies on four key hyperparameters that control the edge selection, expansion timing, and pruning processes. In the following, we describe the key components and their dataset-specific configurations (Table B).

expansion_threshold determines the threshold for selecting edges for expansion, based on the gradient-based importance metric g_{abs} (Section 3.1). For complex tasks requiring larger neural networks, such as those in the Unseen NAS Datasets [13], we used lower values of `expansion_threshold` (approximately 0.1) to allow more edges to be selected. For simpler tasks, such as MNIST, higher values (around 0.7) were employed to limit expansion and maintain model compactness.

pruning_threshold defines the threshold for edge deletion during pruning, also based on the g_{abs} metric. A value of 0.1 was used in all experiments, as it effectively removed edges with minimal impact on model performance. This value was found to balance the reduction in model size with the preservation of task-specific accuracy, allowing up to 40% of newly added edges to be pruned without significant performance degradation (Section 3.3.2).

plateau_window_size defines the number of epochs during which the validation loss history is analyzed to determine when expansion events occur. A fixed value of 5 epochs was used, as it provided a robust window to assess whether the validation loss had stabilized. This choice ensured timely architecture expansion while avoiding unreasonable modifications.

plateau_threshold controls the sensitivity of the expansion trigger by specifying the maximum average absolute change in validation loss. The network is considered to have plateau if the performance, measured over a sliding window of length `plateau_window_size`, shows no meaningful improvement. When this condition is met, SSONN initiates a structural expansion of the architecture. Intuitively, lower values of `plateau_threshold` delay expansion, requiring more stable convergence before modifying the architecture. Conversely, higher values permit earlier expansion, useful in tasks with noisy validation dynamics or high variance in early training.

Table 4: Dataset-specific hyperparameters for SSONN's dynamic expansion. The `expansion_threshold` controls edge selection via gradient-based importance, `plateau_threshold` determines expansion timing based on validation loss stability.

Dataset	`expansion_threshold`	`plateau_threshold`
AddNIST	0.30	0.05
Chesseract	0.05	0.50
CIFArTile	0.10	0.05
GeoClassing	0.40	0.50
Gutenberg	0.05	0.05
Language	0.20	0.05
MultNIST	0.30	0.005
MNIST	0.70	0.005
FashionMNIST	0.50	0.05

C Analytical Justification

Edge sensitivity. Let $L : \mathbb{R}^d \to \mathbb{R}$ be a differentiable loss function, E is a set of edges, $e \in E$, $w_e \in \mathbb{R}$ is the weight of e, and $x \in X$ is a sample x in training set X. If we assume

- that in global minima of L $\forall e, x$ we have $g_{\text{abs}}(e, x) = 0$, where g_{abs} is defined following formula 1,
- for L, if $\left|\frac{\partial L(x_2)}{\partial x}\right| > \left|\frac{\partial L(x_1)}{\partial x}\right|$, $x_1 \neq x_2$, then $L(x_2) > L(x_1)$.

then the replacement of the edge with the largest $\sum_x g_{\text{abs}}(e, x)$ in local minima of L is the optimal way to decrease loss by network expansion.

Let's define $e_1 \in E$ with associated weights $w_1(e)$ and $e_2 \in E$ with associated weights $w_2(e)$, and $w_1 \neq w_2$. If $\sum_{x,e_1} g_{\text{abs}}(e_1, x) > \sum_{x,e_2} g_{\text{abs}}(e_2, x)$ then $\sum_x L(e_1, x) > \sum_x L(e_2, x)$.

Let's split all the edges to k subgroups, each subgroup is related to sublayer. Loss derivative is computed via chain rule and is a product: $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial w(e^k)} \cdot \frac{\partial w(e^k)}{\partial w(e^{k-1})} \cdot \ldots \cdot \frac{\partial w(e^2)}{\partial w(e^1)} \cdot \frac{\partial w(e^1)}{\partial x}$

Using the property if $|\prod_k a_k| > |\prod_k b_k|$ then $\sum_k |a_k| > \sum_k |b_k|$ and our assumption about properties of the L it's easy to show correctness of the lemma.

We select the edge (e) with the maximum sum $\sum_x g_{\text{abs}}(e, x)$, as it has the greatest contribution to the loss gradient.

Adding parameters to such an edge maximizes the reduction of (L), since:

$$\Delta L \propto \sum_x g_{\text{abs}}(e, x).$$

From Lemma, it follows that replacing an edge with a greater sum of gradients leads to a larger reduction in (L).

Computational and memory cost of the gradient-based metrics. Let $B = |\textit{dataloader}|$ be the number of mini-batches processed, E — the number of learnable weights (edges) in the target layer, C_{fb} — the computational cost of one forward plus backward pass through the network on a single mini-batch.

Following both Equation 1 and Equation 2, algorithm steps are:

1. one forward+backward pass per batch,
2. one element-wise addition of two E-length tensors per batch,
3. a single post-loop reduction and normalization over E elements.

Hence the *time* complexity is

$$T_{g_{\text{abs}}} \;=\; \mathcal{O}\big(B\,(\,C_{\text{fb}} + E\,)\big) \;=\; \mathcal{O}\big(B\,C_{\text{fb}}\big), \tag{8}$$

because $C_{\mathrm{fb}} \gg E$ in most settings.

The *additional memory* is dominated by the accumulated values in tensors: $M_{g_{\mathrm{abs}}} = \mathcal{O}(E)$ for both mean and variance metrics. No other step stores intermediate tensors larger than the edge-metric itself, so the overall RAM overhead is linear in the layer size and independent of the number of batches.